

Unit-1: Introduction to Basics of Programming: Reasons for Studying Concepts of Programming Languages; Programming Paradigms – Imperative, Object-Oriented, Logical, Functional; Compilation vs. Interpretation – How Programs Run; Memory Management Basics – Stack vs. Heap Allocation in C; The general problem of describing syntax, Formal methods of describing syntax : Basic Syntax and Semantics of BNF (Terminal, Non Terminal symbols and Production Rules); Names, Variables, Concept of Binding, Scope and Lifetime; Scalar Data Types - Primitive Types and Character, String types and user defined ordinal types; Basic Vector Data Types - Arrays and Indices, Array Initialization, Rectangular and Jagged Arrays and Implementation of Array Types;

1.0 Introduction to Principles of Programming Languages:

1.0.1 What is a Programming Language?

A programming language is a set of rules and symbols that allows humans to communicate with computers. Just as we use language like English to communicate with each other, we use programming languages to tell computers what to do.

Example: When a programmer writes a piece of code in Python, they use specific keywords and syntax that the Python interpreter can read and execute.

1.0.2 Historical Perspective and Evolution:

The history of programming languages dates back to the early days of computers. In the 1940s, when the first electronic computers were developed, programming was done using machine code, which consists of binary numbers (0s and 1s). This was complex and required a deep understanding of hardware.

In the 1950s, the first assembly languages were created. These languages allowed programmers to use symbolic names instead of binary code. For instance, instead of writing a series of 0s and 1s, a programmer could use commands like 'ADD' for addition or 'SUB' for subtraction. While this was an improvement, assembly languages were still specific to a particular type of computer hardware.

The 1960s saw the rise of high-level programming languages. For example, Fortran (short for Formula Translation) was introduced for scientific and engineering calculations, while COBOL (Common Business Oriented Language) was designed for business applications. These languages were more user-friendly and allowed programmers to focus on solving problems rather than dealing with hardware details.

In the following decades, many new programming languages emerged. In the 1970s, C was developed, providing a powerful way to write system software. The 1980s and 1990s brought languages like C++ and Java, which introduced object-oriented programming, allowing for a more modular and reusable approach to software design.

Today, programming languages continue to evolve, with new languages like Python, Ruby, and JavaScript gaining popularity due to their simplicity and effectiveness in web development and data analysis.

1.0.3 The Importance of Programming Languages:

Programming languages are essential because they make it possible to create software applications, websites, games, and much more. Without programming languages, computers would not be able to perform tasks or solve problems for us.

1.0.4 Categories of Programming Languages:

Programming languages can be grouped into different categories based on their features and use cases. Here are some of the main categories:

1. High-Level vs. Low-Level Languages:

- **High-Level Languages:** These languages are closer to human languages and further from machine code. They are easier to read and write. Examples include Python, Java, and C#.
- **Low-Level Languages:** These languages are closer to machine code, which is the language understood by computers. They are harder to read and write. Examples include Assembly language and Machine code.

Example:

Imagine writing a letter to a friend in English (high-level language) versus writing instructions for a robot in binary code (low-level language). The English letter is easier to understand, while the binary code is more precise but difficult for humans to read.

2. Procedural vs. Object-Oriented Languages:

- **Procedural Languages:** These languages follow a sequence of commands or procedures. They focus on the steps needed to perform a task. Examples include C and Pascal.
- **Object-Oriented Languages:** These languages use objects to represent data and methods. They focus on the relationships between objects. Examples include Java and C++.

Example:

Think of procedural programming like following a recipe step-by-step to bake a cake. You need to follow each step in order. In contrast, object-oriented programming is like having a kitchen full of different tools and ingredients (objects) that can all interact with each other in various ways.

1.0.3 Key Factors in Language Design:

When designing a programming language, several key factors are considered to ensure its effective and user-friendly. These factors include:

1. Syntax:

Syntax refers to the set of rules that defines how programs are written in a particular language. Just as proper grammar is important in English, correct syntax is necessary in programming.

Example:

In Python, the syntax for printing a message is:

```
```python
print("Hello, World!")
```
```

If you forget the parentheses or quotes, the program will not run correctly.

2. Semantics:

Semantics refers to the meaning of the instructions in a programming language. It is important to understand not only how to write code but also what it does.

Example:

In the statement:

```
```python
x = 5
```
```

The syntax is correct, but the semantics tell us that we are assigning the value 5 to the variable x.

3. Abstraction: Abstraction is the concept of hiding complex details and exposing only the necessary parts. High-level languages provide a greater level of abstraction, allowing programmers to use complex functionalities without needing to know how they work internally. For example, when using a database, a programmer can use simple commands to perform operations without needing to understand the underlying data storage mechanisms.

4. Efficiency: A programming language should be efficient in terms of performance and resource usage. This includes how quickly a program runs and how much memory it consumes. Languages like C and C++ are known for their efficiency, making them suitable for systems programming.

5. Portability: A good programming language should allow programs to run on different types of hardware and operating systems with minimal changes. Java, for instance, is designed to be portable with its "write once, run anywhere" capability, meaning code written in Java can run on any device that has a Java Virtual Machine (JVM).

6. Community and Library Support: A programming language is more useful when it has a strong community and a wide range of libraries and frameworks. Languages like Python and JavaScript benefit from large communities that contribute to a wealth of resources, making it easier for programmers to find help and tools for their projects.

1.1 Reasons for Studying Concepts of Programming Languages:

1. To increase the capacity of expressing idea:

- Programmers in the process of developing software, feels difficult to idealize structure either verbally or in writing.
- So, once we learn programming language, we can start expressing idea in a better way than now.

2. To improve the knowledge of selecting an appropriate language:

- Whenever we are working on a project or anything. First thing will be selecting an appropriate language as each and every language has some merits and demerits.
- So, selecting an appropriate language is really important.

3. To increase the ability to learn a new language:

- Now a days, programming languages are in a state of continuous evolution. There are many languages which we never hear about.
- Always be a learner and be ready to learn new languages. As new languages will be advanced than previous.

4. To increase the designing ability of programmers for designing new languages:

- Most professional programmers sometime design languages of one kind or the other to complete their needs and which is easy to learn & most efficient.

5. For better understanding of the importance of implementation:

- The implementation issues that affect a programming language must be considered while learning it. This increases the ability of a programmer to intelligently use a language.
- Many errors can be rectified easily.

6. Overall advancement in computing:

- The popularity of a programming language can be easily determined by the most popular languages are not always the best programming languages.
- **Example:** ALGOL-60 is better control statements than FORTRAN but the problem is ALGOL-60 is difficult to read and understand. Hence programmers failed to find the benefits of ALGOL – 60 over FORTRAN.

1.2 Programming Paradigms – Imperative, Object-Oriented, Logical, Functional:

Programming Paradigms:

Programming paradigms are different ways of thinking about and organizing code. They help programmers decide how to approach a problem and how to design their programs. The four main paradigms are Imperative, Object-Oriented, Logical, and Functional programming. Each has its own rules, techniques, and benefits.

1. Imperative Programming:

Imperative programming is like giving a step-by-step recipe to a chef. The programmer tells the computer exactly what to do and how to do it. This includes giving commands to change the program's state.

Features: Uses variables, loops, and conditionals to control execution.

Example:

```
```python
Imperative example in Python
total = 0
for i in range(1, 6): # Loop from 1 to 5
 total += i # Add each number to total
print(total) #
```
```

In this example, the program tells the computer to start with a total of 0, then add each number from 1 to 5 to that total. The focus is on how to achieve the result step by step.

Typical Use: System programming, algorithms, and low-level hardware interactions.

2. Object-Oriented Programming (OOP):

Object-Oriented Programming is like organizing a school. Each student can be thought of as an object that has properties (like name and age) and behaviors (like studying or taking tests). OOP allows programmers to create objects that represent real-world things.

Features: Encapsulation, inheritance, polymorphism, and abstraction.

Example:

```
```python
class Student:
 def __init__(self, name, age):
 self.name = name
```

```
self.age = age
def study(self):
 return f'{self.name} is studying.'
Creating an object
student1 = Student("Alice", 20)
print(student1.study()) # Output: Alice is studying.
'''
```

In this example, we define a `Student` class with properties and a method. The object `student1` represents a specific student, Alice. OOP helps organize code into reusable parts.

**Typical Use:** Software design, GUI applications, simulations, and enterprise systems.

### 3. Logical Programming:

Logical programming is like solving a puzzle. Instead of telling the computer how to solve a problem, the programmer states facts and rules. The computer figures out the solution based on these facts.

**Features:** Uses facts, rules, and queries to derive solutions.

#### Example:

```
'''prolog
% Logical example in Prolog
parent(john, mary).
parent(mary, sam).
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
% Query
?- grandparent(john, sam).
'''
```

In this example, we define relationships between people. The query asks if John is a grandparent of Sam. The computer uses the facts and rules to find the answer. This paradigm is often used in artificial intelligence.

**Typical Use:** AI, expert systems, automated reasoning, and knowledge representation.

### 4. Functional Programming:

Functional programming is like using math to solve problems. Instead of changing states or using loops, it focuses on using functions to process data. Each function takes inputs and produces outputs without changing anything outside of itself.

**Features:** Uses pure functions, higher-order functions, recursion, and immutability.

**Example:**

```
```haskell
-- Functional example in Haskell

sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs

-- Using the function

result = sumList [1, 2, 3, 4, 5] -- Output: 15
```
```

In this example, `sumList` is a function that takes a list of numbers and returns their sum. It does not change the list; instead, it uses recursion to compute the result. Functional programming emphasizes immutability and pure functions.

**Typical Use:** Concurrent programming, distributed systems, data transformations, and mathematical modeling.

## 1.3 Compilation vs. Interpretation – How Programs Run:

When we write a computer program, we use a programming language to give instructions to the computer. However, computers do not understand these high-level languages directly. Instead, they need to convert these instructions into a language they can understand, called machine code. There are two main ways to do this: compilation and interpretation. In this section, we will explore both methods and how they affect the way programs run.

### 1.3.1 What is Compilation?

Compilation is the process of converting the entire source code of a program into machine code before the program runs. This is done using a program called a compiler. The compiler takes the high-level code we write and translates it into a complete machine code file.

**How Compilation Works:**

1. **Writing Code:** First, a programmer writes the source code in a high-level language, such as C or Java.
2. **Running the Compiler:** The programmer then runs the compiler on this source code. The compiler checks the code for errors and translates it all at once into machine code.
3. **Creating an Executable:** After the compilation process, the compiler generates an executable file. This file can be run directly by the computer.

**Example of Compilation:**

For example, consider a simple program written in C:

```
```c
#include <stdio.h>
```

```
int main() {  
    printf("Hello, World!\n");  
    return 0;  
}  
...
```

When this code is compiled, the compiler produces a machine code file that the computer can execute. The entire translation happens before the program runs.

Advantages of Compilation:

- **Speed:** Once compiled, the program runs very quickly because it is already in machine code.
- **Optimization:** Compilers can optimize the code, making it run more efficiently.

Disadvantages of Compilation

- **Longer Development Time:** If there are errors in the code, the programmer must recompile the entire program to see the changes.
- **Platform Dependency:** The compiled program may only run on the specific type of computer (architecture) it was compiled for.

1.3.2 What is Interpretation?

Interpretation is a different method where the source code is translated into machine code one line at a time, while the program is running. This is done using a program called an interpreter.

How Interpretation Works:

1. **Writing Code:** A programmer writes the source code in a high-level language, like Python or Ruby.
2. **Running the Interpreter:** The interpreter reads the code line by line. It translates each line into machine code and executes it immediately.

Example of Interpretation:

Using the same example, here is a simple program written in Python:

```
```python  
print("Hello, World!")
...
```

When this code runs, the interpreter reads the line, translates it to machine code, and then executes it right away.

### Advantages of Interpretation:

- **Ease of Debugging:** Programmers can test their code line by line, making it easier to find errors.



- **Platform Independence:** The same code can run on any computer that has the appropriate interpreter.

## Disadvantages of Interpretation:

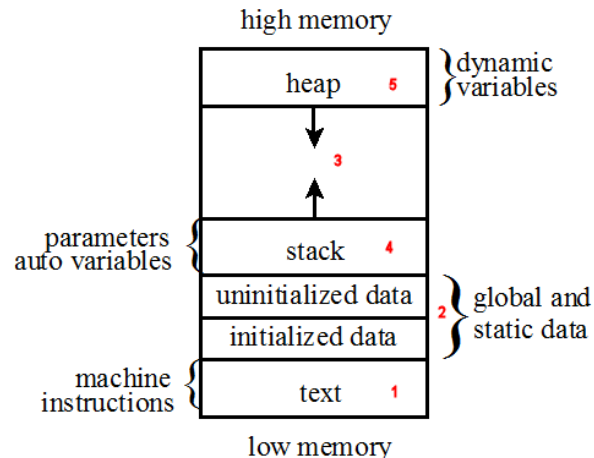
- **Slower Execution:** Since the code is translated line by line during execution, interpreted programs often run slower than compiled ones.
- **Less Optimization:** Interpreters do not usually optimize the code as much as compilers do.

## Compilation Vs Interpretation:

Feature	Compilation	Interpretation
Translation Method	Entire code is translated at once	Line by line translation
Execution Speed	Fast (after compilation)	Slower (during execution)
Development Time	Long (due to recompilation)	Shorter (easily debugging)
Platform Dependency	Often platform-dependent	Generally, Platform independent

## 1.4 Memory Management Basics – Stack vs. Heap Allocation in C:

Memory management is a crucial aspect of programming in C. Understanding how memory is allocated and managed can help programmers write efficient and effective code. In C, memory allocation occurs mainly in two areas: the stack and the heap.



### 1. The Stack:

The stack is a section of memory that stores temporary data. It is used for function calls and local variables. When a function is called, a new block of memory is created on the top of the stack for that function. This block is called a stack frame. When the function finishes executing, its stack frame is removed from the stack. This is often referred to as "popping" off the stack.

### Characteristics of the Stack:

1. **Automatic Allocation:** Memory on the stack is automatically allocated and deallocated. When a function is called, memory is allocated, and when it returns, the memory is freed.

2. **Limited Size:** The stack has a limited size. If too much memory is used, it can lead to a stack overflow, causing the program to crash.
3. **Fast Access:** Accessing data on the stack is very fast because it follows the Last In, First Out (LIFO) principle.

### Example of Stack Allocation:

Here is a simple example of stack allocation in C:

```
```c
#include <stdio.h>

void function() {
    int localVar = 10; // This variable is stored in the stack
    printf("Local Variable: %d\n", localVar);
}

int main() {
    function(); // When function is called, localVar is allocated on the stack
    // When function returns, localVar is automatically freed
    return 0;
}
```
```

In this example, the variable `localVar` is allocated on the stack when `function` is called. Once `function` completes, `localVar` is removed from the stack.

## 2. The Heap:

The heap is a section of memory used for dynamic memory allocation. Unlike the stack, memory in the heap must be managed manually by the programmer. This means that the programmer must allocate and deallocate memory as needed.

### Characteristics of the Heap:

1. **Dynamic Allocation:** Memory on the heap can be allocated during the runtime of the program. This allows for more flexibility in managing memory.
2. **Larger Size:** The heap is typically much larger than the stack, allowing for the allocation of larger blocks of memory.
3. **Slower Access:** Accessing data on the heap is slower compared to the stack because it involves more complex management.

### Example of Heap Allocation:

```
```c
#include <stdio.h>
```

```
#include <stdlib.h> // Required for malloc and free

int main() {
    int *ptr = (int*)malloc(sizeof(int)); // Allocate memory on the heap
    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1; // Exit if memory allocation fails
    }

    *ptr = 20; // Set the value at the allocated memory
    printf("Value: %d\n", *ptr);

    free(ptr); // Free the allocated memory

    return 0;
}
```

In this example, we use `malloc` to allocate memory on the heap for an integer. We must also use `free` to release that memory when it is no longer needed. If we forget to free the memory, it can lead to memory leaks, which can consume resources and slow down the program.

Key Differences Between Stack and Heap:

Feature	Stack	Heap
Allocation Method	Automatic	Manual
Size	Limited	Larger
Speed	Fast	Slower
Lifetime	Function scope	Until explicitly freed

1.5 The general problem of describing syntax, Formal methods of describing syntax: Basic Syntax and Semantics of BNF (Terminal, Non-Terminal symbols and Production Rules)

1.5.1 The General Problem of Describing Syntax:

Syntax refers to the rules that define the structure of statements in a programming language. Just as grammar sets the rules for sentence construction in human languages, syntax provides guidelines for how code must be written so that a computer can understand it. The general problem of describing syntax arises from the need to clearly and accurately represent these rules so that both humans and machines can interpret programming languages consistently.

Importance of Syntax:

Understanding syntax is crucial for several reasons:

1. **Clarity:** Well-defined syntax helps programmers write clear and understandable code.

2. **Error Detection:** Proper syntax allows for easier identification of mistakes in code.
3. **Communication:** A common syntax enables programmers to share ideas and code without confusion.

1.5.2 Formal Methods of Describing Syntax:

To address the general problem of describing syntax, formal methods are used. These methods provide precise languages for defining the syntax of programming languages. The most common formal methods include:

1. Context-Free Grammars (CFG):

A context-free grammar is a set of rules that describe how to form valid strings in a language. A CFG consists of 4-tuple: $G = (V, T, P, S)$

- **Non-terminals (V):** Symbols that can be replaced by groups of terminals and/or other non-terminals.
- **Terminals (T):** The basic symbols from which strings are formed (e.g., keywords, operators).
- **Productions (P):** Rules that define how non-terminals can be replaced by combinations of terminals and non-terminals.
- **Start Symbol (S):** A special non-terminal from which the generation of strings begins.

Example:

Consider a simple programming language that supports arithmetic expressions. A CFG for this language could be:

1. $E \rightarrow E + T \mid E - T \mid T$
2. $T \rightarrow T * F \mid T / F \mid F$
3. $F \rightarrow (E) \mid \text{number}$

In this grammar:

- E is an expression.
- T is a term.
- F is a factor.

number is a terminal representing numeric values.

Using this grammar, the string " $3 + 4 * (2 - 1)$ " can be generated by the above production rules.

2. Backus-Naur Form (BNF):

Backus-Naur Form is a notation used to express context-free grammars. It simplifies the representation of CFGs and is widely used in programming language specifications.

A **BNF rule** is written as:

$\langle \text{non-terminal} \rangle ::= \text{expression}$

- $::=$ means "is defined as"
- The left side is a non-terminal (written inside $\langle \rangle$)

- The right side is a sequence of terminals and/or non-terminals

Example:

Using BNF, the previous grammar can be represented as:

$\langle \text{expression} \rangle ::= \langle \text{expression} \rangle + \langle \text{term} \rangle$

$| \langle \text{expression} \rangle - \langle \text{term} \rangle$

$| \langle \text{term} \rangle$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle * \langle \text{factor} \rangle$

$| \langle \text{term} \rangle / \langle \text{factor} \rangle$

$| \langle \text{factor} \rangle$

$\langle \text{factor} \rangle ::= (\langle \text{expression} \rangle)$

$| \text{number}$

Here, angle brackets denote non-terminals, while the symbols on the right side represent the possible expansions.

3. Abstract Syntax Trees (AST):

An Abstract Syntax Tree is a tree representation of the abstract syntactic structure of source code. Each node in the tree represents a construct occurring in the source code. ASTs abstract away certain syntactic details to focus on the logical structure.

Example:

For the expression "3 + 4 * (2 - 1)", the AST would look like this:

```

      +
     /\
    3 *
     /\
    4 -
     /\
    2 1
  
```

In this tree:

- The root node represents the addition operation.
- Sub-nodes represent the operands and the nested operations.

1.5.3 Basic Syntax and Semantics of BNF:

BNF is a notation used to express the grammar of a language. It consists of two main types of symbols: terminal symbols and non-terminal symbols.

1. Terminal Symbols:

Terminal symbols are the basic symbols from which strings (sentences) in the language are formed. They are the actual characters or tokens that appear in the language. For example, in a programming language, terminal symbols might include keywords like 'if', 'else', 'while', or symbols like '+', '-', '*', and '/'.

Example:

In the expression 'if (x > 0)', the terminal symbols are 'if', '(', 'x', '>', '0', and ')'.

2. Non-Terminal Symbols:

Non-terminal symbols are the syntax represent groups of terminal symbols and can be thought of as placeholders for patterns or structures in the language. They help define the rules for how terminal symbols can be combined. Non-terminal symbols are usually written in angle brackets, like '<expression>' or '<statement>'.

Example:

In a simple language, we might define a non-terminal symbol '<condition>' that consists of a variable, a comparison operator, and a number:

```
...  
<condition> ::= <variable> <operator> <number>  
...
```

3. Production Rules:

Production rules describe how non-terminal symbols can be replaced with combinations of terminal and non-terminal symbols. Each rule provides a way to construct valid strings in the language.

Example:

Consider the following production rules in BNF:

```
...  
<statement> ::= <if-statement> | <while-statement>  
<if-statement> ::= if <condition> then <statement>  
<while-statement> ::= while <condition> do <statement>  
<condition> ::= <variable> <operator> <number>  
<operator> ::= > | < | == | !=  
<variable> ::= x | y | z
```

<number> ::= 0 | 1 | 2 | ... | 9

...

In this example:

- The '<statement>' can either be an '<if-statement>' or a '<while-statement>'.
- An '<if-statement>' begins with the keyword 'if', followed by a '<condition>', and then the keyword 'then' and another '<statement>'.
- The '<condition>' can be formed by a '<variable>', an '<operator>', and a '<number>'.

1.6 Names, Variables, Concept of Binding, Scope and Lifetime:

1. Names:

- In programming, a name is a label we use to identify a piece of data or a function. Think of names like the labels on boxes in your room. Each box has a name that tells you what is inside.
- **For example**, if you have a box labeled "Toys," you know that it contains toys. In the same way, in programming, we use names to refer to data.
- In most programming languages, names can consist of letters, numbers, and sometimes special characters, but they usually cannot start with a number.
- **For example**, 'age', 'total_score', and 'item1' are valid names.

2. Variables:

- A variable is a type of name that holds a value. It acts like a container that can store data temporarily while a program runs. Just as you can put different items in a box, you can change the value stored in a variable.
- **For example**, in Python, you can create a variable called 'age' and assign it a value:

```
```python
age = 10
```
```

Here, the name 'age' is used to refer to the value '10'. You can later change the value:

```
```python
age = 12
```
```

Now, the variable 'age' holds the value '12' instead of '10'.

3. Concept of Binding:

Definition: It refers to the process of associating identifiers (like variables, functions, or objects) with their corresponding values or references.

- The time at which a binding takes place is called binding time.

- Bindings can take place at language design time, language implementation time, compile time, load time, link time, or run time.
 - For example, the asterisk symbol (*) is usually bound to the multiplication operation at language design time.
 - A data type, such as int in C, is bound to a range of possible values at language implementation time.
 - At compile time, a variable in a Java program is bound to a particular data type.
 - A variable may be bound to a storage cell when the program is loaded into memory.
 - A call to a library subprogram is bound to the subprogram code at link time.

Types of bindings:

1. Static Binding (Early Binding):

- A binding is static if it first occurs before run time and remains unchanged throughout program execution.
- If static, the type may be specified by either an explicit or an implicit declaration.
 - An explicit declaration is a program statement used for declaring the types of variables.
 - An implicit declaration is a default mechanism for specifying types of variables (the first appearance of the variable in the program).
 - FORTRAN, PL/I, BASIC, and Perl provide implicit declarations
 - **Advantage:** Enables compile-time checking
 - **Disadvantage:** reliability (less trouble with Perl)

2. Dynamic Binding:

- A binding is dynamic if it first occurs during execution or can change during execution of the program.
- Specified through an assignment statement. example: JavaScript
 - list = [2, 4.33, 6, 8];
 - list = 17.3;
- **Advantage:** flexibility (generic program units)
- **Disadvantages:**
 - Higher runtime cost (dynamic type checking and interpretation)
 - Type error detection by the compiler is difficult

4. Scope:

Scope defines where a name can be accessed or used in a program. It is important to know the scope of a variable to understand where you can use it. There are generally two types of scope: local and global.

a. Static Scope:

- Static scope, also known as lexical scope, refers to the region of the program where a variable is defined and can be accessed.
- In static scoping, the scope of a variable is determined at compile time, based on the structure of the code.

- This means that a variable's scope is fixed and does not change at runtime.

Characteristics:

- The definition of a variable is resolved by searching its containing block or function. If that fails, then searching the outer containing block and so on.
- A variable refers to the nearest enclosing definition in the textual code.
- Most modern programming languages (C, C++, Java, Python) use static scope.

Example: Static scope code in C

```
#include <stdio.h>

int a = 50, b = 60;

void swap() {
    int t;
    t = a;
    a = b;
    b = t;
    printf("Inside swap(): a = %d, b = %d\n", a, b);
}

int main() {
    int a = 10, b = 20; // Local to main, not used by swap
    swap();
    printf("Inside main(): a = %d, b = %d\n", a, b);
    return 0;
}
```

Output: Inside swap(): a = 60, b = 50

Inside main(): a = 10, b = 20

b. Dynamic scope:

- Dynamic scope means that the scope of a variable is determined at runtime, based on the call stack.

Key Characteristics:

- The definition of a variable is resolved by searching its containing block and if not found, then searching its calling function and if still not found then the function which called that calling function will be searched and so on.
- Rare in modern languages, but seen in some older or scripting languages like early Lisp, Bash, or Perl (optional).

Example: Dynamic scope (using function parameters in C)

Since C doesn't support dynamic scope directly, we simulate it by passing variables as parameters.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t;
    t = *a;
    *a = *b;
    *b = t;
    printf("Inside swap(): a = %d, b = %d\n", *a, *b);
}

int main() {
    int a = 10, b = 20; // Local variables passed to swap
    swap(&a, &b);
    printf("Inside main(): a = %d, b = %d\n", a, b);
    return 0;
}
```

Output: Inside swap(): a = 20, b = 10

Inside main(): a = 20, b = 10

5. Lifetime:

- Lifetime describes how long a variable exists in memory. A variable's lifetime begins when it is created and ends when it is destroyed or goes out of scope.

1. **Local variables:** These variables have a lifetime that lasts only as long as the function in which they were created is running. Once the function ends, the local variables are removed from memory.

Example:

```
```python
def temporary_variable():
 z = 30 # z is created when the function starts
 print(z)
temporary_variable() # z exists and prints 30
print(z) # This would cause an error because z no longer exists
```

'''

2. **Global Variables:** Global variables, however, exist as long as the program is running. They can be used by any function throughout the program until the program ends.

**Example:**

```
'''python
a = 50 # a is a global variable

def use_global():
 print(a) # a can be accessed here

use_global() # This will print 50 '''
```

## 1.7 Scalar Data Types - Primitive Types and Character, String types and user defined ordinal types:

### 1.7.1 Scalar Data Types:

Scalar data types are the simplest types of data in programming. They represent a single value and are used to store basic information. Scalar types can be divided into three main categories: primitive types, character types, string types, and user-defined ordinal types.

#### 1. Primitive Types:

Primitive types are the basic building blocks of data in programming. They represent simple values and usually correspond to basic data types provided by most programming languages. The most common primitive types include:

##### a. Integer:

An integer is a whole number that can be positive, negative, or zero. Integers do not have any decimal points.

**For example:**

- 5
- -3
- 0

##### b. Floating Point:

A floating-point number, or float, is a number that can have a decimal point. This type is used for more precise values that cannot be represented as whole numbers.

**For example:**

- 3.14
- -0.001
- 2.0

##### c. Boolean:

A boolean type can hold one of two values: true or false. This type is often used for decision-making in programs.

**For example:**

- true

- false

## 2. Character Types:

A character type represents a single character. This can be a letter, a number, a symbol, or even a space. In most programming languages, characters are enclosed in single quotes.

**For example:**

- 'A'
- '7'
- '!'

**Example:** If you want to store the letter "B" in your program, you would use the character type like this: ``char letter = 'B';``.

## 3. String Types:

A string type is a sequence of characters. Strings can hold words, sentences, or any combination of characters. Strings are usually enclosed in double quotes.

**For example:**

- "Hello, world!"
- "12345"
- "Goodbye!"

**Example:** If you want to store the name of a person, you would use a string type. For instance, ``String name = "Alice";``.

## 4. User-Defined Ordinal Types:

User-defined ordinal types are types created by the programmer. They allow for the definition of new data types that can be used in a program. This includes enumerations and other types defined based on existing scalar types.

### a. Enumerations:

An enumeration, or enum, is a user-defined type that consists of a set of named values. It is often used when a variable can only have a specific set of values.

**For example:**

```
```plaintext
```

```
enum Days {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday}
```

```
```
```

**In this example,** ``Days`` is a user-defined ordinal type that represents the days of the week. You can use this type to create variables that only take on one of these specific values.

**Example:** If you have an enum for ``Days``, you can declare a variable like this: ``Days today = Days.Monday;``.

### **b. Other User-Defined Types:**

In addition to enumerations, programmers can create other user-defined types, such as records or classes, that can include primitive types, characters, strings, and even other user-defined types. This allows for greater flexibility and organization in programming.

## **1.8 Basic Vector Data Types - Arrays and Indices, Array Initialization, Rectangular and Jagged Arrays and Implementation of Array Types:**

### **1.8.1 Basic Vector Data Types:**

In computer programming one common way to store a collection of data is through the use of vectors. Vector data types, commonly known as arrays, are essential structures in programming that allow us to store multiple values in a single variable.

### **1.8.2 Arrays and Indices:**

An array is a data structure that holds a fixed a number of values of the same type. Think of an array as a row of boxes, where each box can store one value.

Each box in the array is called an element, and each element has a specific position known as its index.

#### **Example of Array and Indices:**

Consider this example of an array that holds five numbers:

```
```plaintext
numbers = [10, 20, 30, 40, 50]
```
```

In this array, the numbers are stored in the following way:

```
`numbers[0]` is 10
`numbers[1]` is 20
`numbers[2]` is 30
`numbers[3]` is 40
`numbers[4]` is 50
```

The index starts from 0, meaning that the first item is at index 0, the second at index 1, and so on. This is an important point to remember when working with arrays.

### **1.8.3 Array Initialization:**

Initializing an array means creating an array and assigning values to it. There are several ways to initialize an array.

#### **Example of Initialization:**

**1. Static Initialization:** This is when you assign values directly at the time of declaration.

```
```plaintext
int[] numbers = {10, 20, 30, 40, 50};
```
```

**2. Dynamic Initialization:** This is when you first create the array with a specific size and then assign values to each index.

```
```plaintext
int[] numbers = new int[5]; // Creates an array of size 5
numbers[0] = 10; // Assigns 10 to the first index
numbers[1] = 20; // Assigns 20 to the second index
numbers[2] = 30; // Assigns 30 to the third index
numbers[3] = 40; // Assigns 40 to the fourth index
numbers[4] = 50; // Assigns 50 to the fifth index
```
```

There are different ways to initialize an array, depending on the programming language.

### Example of Initialization:

**a) In Python, you can initialize an array like this:**

```
ages = [25, 30, 22, 28, 35] # Initializing an array with values
```

**b) In other languages like C++, like this:**

```
int ages[5] = {25, 30, 22, 28, 35}; // Initializing an array with values
```

**c) You can also create an array without initial values, where the values are set later:**

In python

```
Initializing an empty array
```

```
ages = [0] * 5 # This creates an array of 5 elements, all set to 0.
```

### 1.8.4 Rectangular Arrays:

A rectangular array is a two-dimensional array where every row has the same number of columns. This means that each row is structured similarly, creating a grid-like format.

### Example of a Rectangular Array:

In python

```
A 2D rectangular array
```

```
matrix = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
]
```

In this example, `matrix[0][0]` is 1, `matrix[1][2]` is 6, and so on. Each row has three columns.

### 1.8.5 Jagged Arrays:

A jagged array, also known as an "array of arrays," allows each row to have a different number of elements. This means that the lengths of the rows do not have to match, creating a more flexible structure.

#### Example of a Jagged Array:

In Python

```
A 2D jagged array
```

```
jagged_array = [
 [1, 2],
 [3, 4, 5],
 [6]
]
```

In this case, the first row has two elements, the second row has three elements, and the third row has only one element.

#### Implementation of Array Types:

When implementing arrays in programming, it is important to choose the type that best fits the needs of your application. Rectangular arrays are useful when you need a fixed-size grid, while jagged arrays provide flexibility when the rows may vary in size. The implementation of arrays may vary depending on the programming language used. Examples in Python and C++.

#### Example Code Snippet: Python Implementation of Rectangular Array

In Python, arrays can be implemented using lists, as shown in previous examples. To access elements, simply use the index:

##### Example1:

```
ages = [25, 30, 22, 28, 35]
print(ages[0])
```

##### Example2:

```
matrix = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
]
print(matrix[1][1])
```



Python also allows you to use libraries like NumPy for more advanced array operations, which are particularly useful for numerical data.

### **Example code snippet:** C++ Implementation of Rectangular Array

In C++, arrays are defined with a specific size. Here's how you can create and access a rectangular array:

```
#include <iostream>

using namespace std;

int main() {
 int matrix[3][3] = {
 {1, 2, 3},
 {4, 5, 6},
 {7, 8, 9}
 };
 cout << matrix[0][1]; // Output: 2
 return 0;}
```

**Example:** Construct a program to create a 2D rectangular array and print the sum of each row.

```
Define the 2D rectangular array (list of lists)
array_2d = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
]

Print the 2D array
print("2D Array:")
for row in array_2d:
 print(row)

print("\nSum of each row:")

Loop through each row and calculate the sum
for i, row in enumerate(array_2d):
 row_sum = sum(row)
 print(f'Sum of row {i + 1}: {row_sum}')
```

**For jagged arrays in C++ implementation, you can use an array of pointers:**

```
#include <iostream>

using namespace std;

int main() {
 int* jagged_array[3];
 jagged_array[0] = new int[2]{1, 2};
 jagged_array[1] = new int[3]{3, 4, 5};
 jagged_array[2] = new int[1]{6};
 cout << jagged_array[1][2]; // Output: 5
 return 0;
}
```

### Assignment Questions:

1. Discuss the importance of studying programming language concepts. How does it help in better software development and language design?
2. Explain the differences between imperative, object-oriented, logical, and functional programming paradigms with examples.
3. Distinguish between compilation and interpretation. How do they impact the way programs run? Support with a suitable example.
4. With examples in C, explain the difference between stack and heap memory allocation. When should each be used?
5. Describe the basic syntax and semantics of Backus-Naur Form (BNF). Define terminals, non-terminals, and production rules with examples.
6. Analyze the concepts of name binding, scope, and lifetime in programming languages. How do they affect program behavior?
7. Explain different scalar data types in programming languages with emphasis on primitive, character, string, and user-defined ordinal types. Provide suitable examples.
8. Differentiate between rectangular and jagged arrays. How are these arrays initialized and implemented in a programming language?